Studio Nx

백엔드 308 — 템포 변경점 모델과 스키마

2026-10-21

백엔드 308 — 템포 변경점 모델과 스키마 (`models/tempo_change.py`, `schemas/tempo.py`)

항목	내용
파일	`studionx/backend/app/models/tempo_change.py`, `studionx/backend/app/schemas/tempo.py`
작업	S1-13 템포 변경: 템포 변경점 표의 모델과 요청, 응답 모양 (PRD 02 FR-202a)
작성	Claude (AI), 2026-10-21, 사용자 검토
이전, 다음	이전 307 오디오 회귀 테스트 실행 명령, 다음 309 템포 서비스

1. 하는 일

곡 중간의 템포 변경점 한 건을 담는 모델(`TempoChange`)과, API가 주고받는 모양(스키마)을 정의합니다. 표 `tempo_changes`는 DB에 처음부터 있었지만(DB.md §3.3, 2026-10-05), 이 표를 읽고 쓰는 모델과 API가 없어서 템포 변경점은 저장도 조회도 할 수 없었습니다.

```
tempo_changes 한 행 (곡 중간의 템포 변경점)
├ project_id  – 프로젝트 (삭제하면 DB가 함께 지움)
├ tick        – 변경 위치. 0보다 커야 함 (0 의 템포는 projects.bpm). 프로젝트 안에서 유일
├ bpm         – 20.00 ~ 300.00
└ ramp        – STEP = 이 위치에서 바로 바뀜, LINEAR = 직전 변경점부터 이 위치까지 서서히 바뀜
```

시작 템포(tick 0)는 `projects.bpm`에 있고, 이 표는 그 뒤의 변경점만 가집니다. 실제 시간(초)은 이 지도로 계산하며, 오디오 엔진(`audio/timing.js`의 `ticksToSecondsWithChanges`)이 읽는 모양이 `{ tick, bpm, ramp }`입니다.

2. 언제 쓰나

쓰는 곳	하는 일
`tempo_service` (309)	목록 읽기와 통째 교체에 모델과 스키마를 씁니다.
`full_project_service` (310)	`GET /projects/{id}/full` 응답의 `tempo_changes`를 `TempoChangeItem`으로 채웁니다.
시험 `test_tempo_routes.py`	요청 검증(422)과 응답 모양을 확인합니다.

3. 코드

3.1 모델

```
class TempoRamp(StrEnum):
    """템포가 바뀌는 방식."""

    STEP = "STEP"
    LINEAR = "LINEAR"
```

```
class TempoChange(Base):
    """템포 변경점. 프로젝트마다 여러 개를 가진다."""
```

```
    __tablename__ = "tempo_changes"

    tempo_change_id: Mapped[int] = mapped_column(BigInteger, primary_key=True, autoincrement=True)
    project_id: Mapped[int] = mapped_column(
        BigInteger, ForeignKey("projects.project_id", ondelete="CASCADE")
    )
    tick: Mapped[int] = mapped_column(INTEGER(unsigned=True))
    bpm: Mapped[Decimal] = mapped_column(Numeric(6, 2))
    ramp: Mapped[TempoRamp] = mapped_column(
        Enum(TempoRamp, native_enum=True), server_default=text("'STEP'")
    )
    created_at: Mapped[datetime] = mapped_column(
        DATETIME(fsp=3), server_default=text("CURRENT_TIMESTAMP(3)")
    )
```

3.2 스키마

```
class TempoChangeInput(BaseModel):
    """템포 줄이 보내는 변경점 한 건."""

    tick: TempoTick
    bpm: TempoBpm
    ramp: Literal["STEP", "LINEAR"] = "STEP"

    @field_validator("bpm")
    @classmethod
    def round_bpm(cls, value: float) -> float:
        """DB 칸(DECIMAL(6,2))에 맞춰 소수 둘째 자리까지만 쓴다."""
        return round(value, 2)
```

```
class TempoChangeItem(TempoChangeInput):
    """응답 변경점 한 건 — TempoChangeInput 에 번호를 더한 모양."""

    tempo_change_id: int
```

```
class TempoChangesSaveRequest(BaseModel):
    """PUT /projects/{id}/tempo-changes — 이 프로젝트의 템포 변경점을 보낸 목록으로 통째 교체."""

    changes: list[TempoChangeInput] = Field(default_factory=list, max_length=TEMPO_CHANGES_MAX)

    @model_validator(mode="after")
    def no_duplicate_ticks(self) -> "TempoChangesSaveRequest":
        """같은 위치에 변경점이 둘이면 거부한다 (DB UNIQUE(project_id, tick) 와 같은 규칙)."""
        ticks = [change.tick for change in self.changes]
        if len(ticks) != len(set(ticks)):
            raise ValueError("tempo changes must have distinct ticks")
        return self
```

```
class TempoChangesResponse(BaseModel):
    """템포 변경점 목록 — tick 오름차순."""

    items: list[TempoChangeItem]
    total: int
```

4. 입력 검증

DB의 `CHECK` 제약(`chk_tempo_bpm`, `chk_tempo_tick`)과 `UNIQUE(project_id, tick)`를 요청 단계에서 먼저 같은 규칙으로 막아, DB 오류(500)가 아니라 422로 알려 줍니다.

규칙	값	어기면
`tick`	1 이상 4294967295 이하	422 (0은 시작 템포의 자리)
`bpm`	20 이상 300 이하, 소수 둘째 자리로 반올림	422 (범위 밖), 반올림은 `round_bpm`이 함
`ramp`	`STEP` 또는 `LINEAR`, 생략하면 `STEP`	422
같은 `tick` 두 건	허용하지 않음	422 (`no_duplicate_ticks`)
목록 길이	500건 이하	422 (실수로 큰 목록을 보내는 것을 막음)

5. 왜 이렇게

선택	이유
응답의 `bpm`을 숫자(`float`)로 보냅니다	오디오 엔진(`timing.js`)이 `typeof c.bpm === 'number'`만 받고 문자열이면 오류를 던집니다. 프로젝트 설정의 `bpm`은 `Decimal`이라 문자열로 나가는데, 그 값을 그대로 쓰면 템포 지도 전체가 거부됩니다.
요청도 `float`으로 받고 둘째 자리로 반올림합니다	화면의 점을 끌면 120.456 같은 값이 나옵니다. DB 칸(`DECIMAL(6,2)`)에 맞춰 서버가 반올림하면 저장된 값과 응답이 어긋나지 않습니다.
요청에 `ramp`를 문자열 `Literal`로 둡니다	DB는 `ENUM('STEP','LINEAR')`이고 모델은 `StrEnum`입니다. 요청은 문자열만 받고 서비스에서 `TempoRamp(...)`로 바꿔서, 알 수 없는 값은 모델에 닿기 전에 422로 막힙니다.
시작 템포(tick 0)를 이 표에 넣지 않습니다	DB 설계(DB.md)가 정한 규칙입니다. 시작 템포는 `projects.bpm` 하나가 맡고, 이 표는 변경점만 가져서 두 곳이 어긋날 일이 없습니다.
모델에 `relationship`을 두지 않았습니다	프로젝트 삭제는 DB의 `ON DELETE CASCADE`가 처리하고, 읽기와 쓰기는 `project_id`로 직접 조회합니다. `Project` 모델을 건드리지 않아도 되고 순환 import도 없습니다.

6. 시험

`studionx/backend/tests/test_tempo_routes.py`의 요청 검증 시험이 확인합니다.

- `tick`이 0, 음수, 문자열이면 422, `bpm`이 19.99나 300.01이면 422, `bpm`이나 `tick`이 없으면 422, 알 수 없는 `ramp`면 422입니다.
- 같은 `tick` 두 건이면 422, 501건이면 422입니다.
- 경계값(`tick` 1, `bpm` 20과 300)은 받아들입니다.
- `bpm` 120.456은 120.46으로 저장되고, 응답의 `bpm`은 모두 숫자입니다.